

Talking-Point Q&A — Detailed Answers

Lowe's Store-Level AI Sales Targeting · Capstone 8 · Group 8 · IIM Calcutta APAL02

Prepared for review · Seven questions answered in technical depth

Question	Page
Q1 · Feature engineering techniques used to decide features	p. 2
Q2 · Standardization & normalization techniques (and why)	p. 4
Q3 · Hyperparameters of each of the 4 models	p. 5
Q4 · Why WAPE? + the final 16 feature importance weights	p. 7
Q5 · How the 5 store-role groups are chosen + thresholds	p. 9
Q6 · Features ignored due to multicollinearity (and why)	p. 11
Q7 · 'subsample 0.8' & 'colsample 0.8' in plain English	p. 13

Q1 • Feature engineering techniques used to decide features

We started with 194 raw columns and narrowed to 16 final features across three iterative rounds. Seven distinct techniques drove every keep/drop decision.

The 7 techniques applied

1. Correlation analysis against target

For every numeric column we computed Pearson correlation with Actual Sales USD. Features with very low absolute correlation (≈ 0) were candidates for removal — but only if no other technique (domain logic, interaction effects) justified keeping them.

Strongest correlations found in lag features ($\text{lag_1} = 0.982$, $\text{roll_4} = 0.981$); weakest in single-decade housing buckets and most distance-band competitor counts (typically $|r| < 0.05$).

2. Multicollinearity check (correlation matrix + sum-to-100 detection)

We computed pairwise correlations between candidate features and flagged any pair with $|r| > 0.85$. We also detected feature groups that summed to $\sim 100\%$ (income-bracket columns, housing-decade columns). These were collapsed into 2-3 strategic engineered features to avoid coefficient instability.

3. Variance / sparsity threshold

Features with near-zero variance — for instance, competitor counts that were 0 for $>95\%$ of stores — were dropped. These cannot help any model split because they don't separate observations.

Examples dropped: Thin Lizzy Hardware (100% zeros), Bobs Supplies (99% zeros).

4. Lag-feature engineering (time-series shift operations)

Per-store grouped shift to create temporal features:

- lag_1 , lag_4 , lag_13 , lag_52 = sales shifted by 1, 4, 13, and 52 weeks per store
- roll_4 , roll_13 = rolling-mean of past 4 / 13 weeks (later dropped — see Q6)

All lag operations used $\text{shift}(1)$ minimum to prevent leakage (the model can never see the week it predicts).

5. Aggregation engineering (combining redundant raw columns)

To preserve signal but avoid multicollinearity, we engineered composite features:

- $\text{income_affluent} = \% \text{ HH } \$150\text{K}-\$250\text{K} + \% \text{ HH } \$250\text{K}+$ (collapses 2 income brackets)
- $\text{housing_new_share} = \% \text{ built } 2000-2009 + \% \text{ built } 2009-2019$ (collapses 2 decade buckets)

- housing_old_share = % built pre-1949 + 1950s + 1960s (collapses 3 decade buckets)
- total_competitor_ta = sum of all 17 competitor counts in trade area (collapses 17 columns to 1)

6. Categorical encoding

Three categorical features were converted to integer codes using sklearn LabelEncoder:

Original	Encoded as	Levels
Urbanicity	Urbanicity_enc	0 = Remote, ..., 6 = Metropolis (7 levels)
CBSA Type	CBSA Type_enc	0 = Non-CBSA, 1 = Micro, 2 = Metro (3 levels)
CBSA Metro Size	CBSA Metro Size_enc	4 ordinal levels

LabelEncoder (not OneHotEncoder) chosen because tree-based models split on numeric thresholds and handle ordinal codes natively without inflating the feature count.

7. Leakage detection (the critical guard)

Three columns had correlations dangerously close to 1.0 — they are 'same-period' measures that would give the model the answer instead of features it has to learn from:

Banned column	Correlation w/ target	Why it's leakage
Plan Sales USD	0.986	It IS the target — what we're trying to predict
Invoice Count	0.972	Same-period transaction count (Sales = Invoices × Avg Ticket)
Avg Ticket	0.133	Same-period decomposition of Sales

A programmatic assertion in feature_engineering.py raises an error if any of these ever enter the feature list.

The funnel

Iteration	Count	What changed
Raw columns	194	All columns in source CSV
After identity/leakage drop	165	Removed Plan Sales, Invoice Count, Avg Ticket, store names, IDs
Phase 1 set (1st round)	40	Business-relevance + correlation + variance threshold
After 1st model iteration	—	XGBoost feature importance + multicollinearity check
Final 16 features	16	Removed dominant lag-only features + redundant aggregates

Q2 • Standardization & normalization techniques (and why)

Short answer

We did NOT apply z-score standardization or min-max normalization to the features fed into our final XGBoost model. This is intentional and correct for the chosen algorithm.

Why no scaling for tree-based models

XGBoost (our primary model) and LightGBM are tree-based ensemble methods. They make splits like:

"if lag_1 > \$900,000 then go right, else go left"

The split threshold (\$900,000) is found by the algorithm itself. The model is invariant to monotonic transformations — multiplying every value by 1000 or taking the log changes no decisions. So scaling adds zero value and adds computational cost.

What we DID do (data preparation)

1. Type conversion

All numeric columns were forced to numeric type using `pd.to_numeric(errors='coerce')` to handle any stray strings or formatting issues in the raw CSV.

2. Missing value handling

Three different strategies were applied based on what NaN means for that column:

- Lag features: leave as NaN (drop rows where any lag is NaN — required because the first few weeks per store have no history)
- Competitor counts: `fillna(0)` — a missing competitor count is treated as 0 competitors
- Income brackets / housing decades: `fillna(0)` — a missing % is treated as 0 share

3. Categorical encoding

LabelEncoder applied to Urbanicity, CBSA Type, CBSA Metro Size (see Q1 for details).

4. Sort ordering (mandatory for lag features)

Before computing lags, the data is sorted by Store ID → Year → Fiscal Week. This ensures `shift(n)` operations correctly reach the previous N weeks of THAT store, not a different store.

Where scaling WOULD matter

If we ran a different algorithm family, scaling becomes necessary:

Algorithm	Needs scaling?	Why
XGBoost (our model)	No	Trees split on thresholds; scale invariant
LightGBM	No	Same as XGBoost
Ridge Regression (our baseline)	Recommended	L2 penalty applies equally to all coefficients — features need comparable scales for fair regularization
Linear regression (OLS)	No (mathematically)	Coefficients adjust to scale, but interpretation is affected
Neural networks	Yes (mandatory)	Gradient descent struggles with unscaled features
K-means / KNN	Yes (mandatory)	Distance metrics dominated by largest-scale feature otherwise

For our Ridge baseline we used sklearn's default fit — coefficients are still interpretable as direction, and the WAPE comparison stays fair because all models score on the same actual sales scale.

Bottom line for the talking point

"Because our chosen model is tree-based (XGBoost), we deliberately did not apply z-score or min-max scaling. Trees are scale-invariant — they make split decisions on raw thresholds, so scaling would add no benefit. What we did was type-coerce, fill missing values context-appropriately, and label-encode categoricals."

Q3 • Hyperparameters of each of the 4 models

All four models trained on the same train/test split (rolling time split, 85th percentile cutoff). Hyperparameters were chosen for cross-model fairness (same learning rate + tree count for both boosting models) and to mirror industry-standard retail forecasting configurations.

1. XGBoost — Primary Production Model

Hyperparameter	Value	What it does
n_estimators	500	Total trees built sequentially
learning_rate	0.05	Each tree's contribution to the ensemble (small = stable convergence)
max_depth	7	Maximum depth of each individual tree
subsample	0.8	Random row sample per tree (see Q7)
colsample_bytree	0.8	Random feature sample per tree (see Q7)
objective	reg:squarederror	Mean-squared-error loss for regression
early_stopping_rounds	50	Stop if validation loss stops improving for 50 rounds
random_state	42	Fixed seed for reproducibility

2. LightGBM — Secondary Benchmark

Hyperparameter	Value	What it does
n_estimators	500	Same as XGBoost for fair comparison
learning_rate	0.05	Same as XGBoost
max_depth	8	Slightly deeper because LightGBM grows leaf-wise
num_leaves	63	Maximum leaves per tree (key LightGBM-specific control)
min_child_samples	50	Minimum samples required in a leaf — prevents tiny over-fit splits
subsample	0.8	Same as XGBoost
colsample_bytree	0.8	Same as XGBoost
objective	regression	Standard regression objective
early_stopping_rounds	50	Same as XGBoost
random_state	42	Same seed for reproducibility

3. Ridge Regression — Linear Baseline

Hyperparameter	Value	What it does
alpha	1.0	L2 regularization strength (default sklearn value)
fit_intercept	True	Allow non-zero intercept
solver	auto	sklearn chooses based on data shape
random_state	42	Reproducibility

Ridge serves as our linear sanity check. If the gradient-boosting models couldn't beat regularized linear regression, the added complexity wouldn't be justified. Ridge scored WAPE 7.11% — very close to XGBoost's 7.05%, telling us the relationship is fairly linear once lag features are present, but the non-linear models still edge it out.

4. Naive Lag-52 Baseline — Zero Hyperparameters

The simplest possible 'model': prediction for week t = actual sales at week $t-52$ (same week last year). There are no parameters to tune. This is the bar every real model must clear to justify its existence. WAPE = 8.27%. Our final model beats it by 15%.

Head-to-head holdout results

Model	WAPE	Bias	R^2	Verdict
XGBoost (selected)	7.05%	-0.97%	0.9769	Best balanced
LightGBM	7.07%	-1.11%	0.9773	Confirms robustness
Ridge	7.11%	+0.39%	0.9783	Strong linear baseline
Naive (lag-52)	8.27%	+0.33%	0.9648	The bar to beat

Q4 • Why WAPE was chosen + the final feature weights

Part A • Why WAPE (not MAPE, RMSE, or MAE)

WAPE = Weighted Absolute Percentage Error. Formula:

$$\text{WAPE} = \sum | \text{Actual} - \text{Predicted} | / \sum \text{Actual} \times 100$$

Plain-English meaning

Of every \$100 of actual sales, how many dollars did the model miss by? Our 7.05% → about \$7 of error per \$100 sold.

Why WAPE beats the alternatives for retail

Metric	Issue	Why WAPE wins
MAPE	Equal weight to a \$1K store and a \$50M store; explodes when actual is near zero	WAPE weights by sales volume — bigger stores' errors count more, matching dollar risk
RMSE	Penalizes large errors quadratically; hard to interpret in dollar terms	WAPE is a single intuitive percentage
MAE (dollars)	No sense of scale — is \$50K error big or small?	WAPE normalizes to % of actual, immediately scale-aware
Bias %	Only direction, not magnitude	WAPE measures magnitude — we report bias separately as a second axis

Industry standard

WAPE is the de-facto reporting metric for retail demand forecasting (used by Walmart, Amazon, Lowe's own legacy planning) precisely because it preserves dollar weighting and yields a single intuitive number.

Part B • The 16 feature weights (XGBoost importance)

These are the relative importance scores XGBoost assigned each feature based on how often and how decisively the feature was used to split data across all 500 trees. Importance is normalized to sum to 100%.

Rank	Feature	Group	Importance %
1	lag_1	Lag / Momentum	62.22%
2	lag_4	Lag / Momentum	29.63%
3	lag_52	Lag / Momentum	3.14%
4	Wallflowers Depot (TA)	Competition	0.70%
5	lag_13	Lag / Momentum	0.55%
6	housing_new_share	Demand – Engineered	0.40%
7	CYE Total Population	Demand – Raw	0.39%
8	CYE Median HH Income	Demand – Raw	0.38%
9	CYE Total Housing Units	Demand – Raw	0.36%

10	income_affluent	Demand - Engineered	0.35%
11	housing_old_share	Demand - Engineered	0.34%
12	CYE HH Density / SqMi	Demand - Raw	0.34%
13	Sister Store Count (TA)	Competition	0.34%
14	total_competitor_ta	Competition	0.33%
15	CYE Total Households	Demand - Raw	0.29%
16	Urbanicity (encoded)	Store / Market	0.24%

How to read the distribution

- Lag features dominate (95.5% combined): lag_1 + lag_4 = 91.85%. This is expected and correct for one-week-ahead forecasting — recent sales are the strongest predictor of next-week sales.
- Market features (housing, income, competition, population) collectively carry ~4% — small but non-zero. They differentiate stores when the lag signal is ambiguous.
- This distribution is healthy: a planner can defend any target by pointing to 'recent run rate, with adjustments for the local market context'.

Q5 • How the 5 store-role groups are chosen + thresholds

The 5 segments and what they mean

Role	What it signals	Planner playbook
High Growth	Strong demand + store outperforming model	Raise targets aggressively
Growth	Moderately positive market signals	Use data-driven target as-is
Neutral	Balanced — no strong signal either way	Hold pattern; standard target
Maintain	Stable demand, moderate competition	Hold targets, monitor closely
Defend	High competition + store underperforming	Conservative targets; protect share

The two composite scores

Each store gets two derived scores from its 16 features:

$$\text{growth_score} = (\text{CAGR_HH} / 2) + (\text{housing_new_share} / 10) - (\text{Bias_pct} / 10)$$

$$\text{risk_score} = (\text{total_competitor_ta} / 50) + (\text{Bias_pct} / 10)$$

$$\text{net_score} = \text{growth_score} - \text{risk_score}$$

What each component captures

- CAGR HH 2010-2020 (positive) → household growth = forward demand expansion
- housing_new_share (positive) → new construction → new-homeowner improvement demand
- Negative bias % → model UNDER-predicts → store is outperforming → upward signal
- total_competitor_ta (positive) → more competition = more downward pressure
- Positive bias % → model OVER-predicts → store is underperforming → downward signal

Why QUANTILE-based thresholds, not fixed cut-offs

Our first iteration used fixed thresholds (e.g., $\text{growth_score} > 1.5$ = High Growth). Result was useless: 86% of stores landed in 'High Growth'. The growth_score distribution was right-skewed and most stores naturally scored above the fixed line.

Switched to quantile-based binning, which guarantees ~20% of stores in each segment regardless of underlying distribution shape.

The thresholds (quantile-based)

Segment	Rule	Stores	% of total
High Growth	$\text{net_score} \geq 80\text{th percentile}$	346	20.0%
Growth	60th to 80th percentile	345	20.0%

Neutral	40th to 60th percentile	345	20.0%
Maintain	20th to 40th percentile	346	20.0%
Defend	Below 20th percentile	345	20.0%

Total: 1,727 stores classified, ~20% in each of 5 segments.

Critical caveat — this is a heuristic, not a learned classifier

Role-of-Store is a rule-based strategic OVERLAY, not a separate trained ML model. The composite scores use hand-chosen weights (divide-by-2 for CAGR, divide-by-10 for housing share, etc.) tuned for interpretability rather than optimality. A future iteration could replace this with a learned clustering algorithm (K-Means on standardized features).

Q6 • Features ignored due to multicollinearity

Categories of dropped features

Category A • Sum-to-100% groups (perfect multicollinearity)

These columns within a group sum to ~100 for every store, so they're mathematically redundant.

Income brackets — 8 columns reduced to 3 strategic shares + median:

Dropped column	Why redundant
CYE % HH Income 25k-35k	Captured by income_low_share aggregate
CYE % HH Income 35k-50k	Mid-low bracket — collapsed into low/diy aggregates
CYE % HH Income 50k-75k	Captured by income_diy_core (dropped in iteration 3)
CYE % HH Income 75k-100K	Captured by income_diy_core (dropped in iteration 3)
CYE % HH Income 100k-150k	Middle income — captured by median + adjacent aggregates
CYE % HH Income 150k-250k	Captured by income_affluent aggregate
CYE % HH Income 250K Plus	Captured by income_affluent aggregate (also 41.6% missing)

Housing decade buckets — 9 columns reduced to median + 2 strategic shares:

Dropped column	Why redundant
CYE % HU Built 1970-1979	Middle decade — captured by Median Year Built
CYE % HU Built 1980-1989	Middle decade — captured by Median Year Built
CYE % HU Built 1990-1999	Middle decade — captured by Median Year Built
CYE % HU Built 2020+	Very small share in most areas — sparse signal

Category B • Perfect inverses

CYE % Housing Units Rented = 100% – CYE % Housing Units Owned. Perfect linear redundancy → one is dropped (we kept % Owned because ownership drives improvement spend).

Category C • Aggregate vs component redundancy

Dropped	Why
CYE Aggregate Family Household Income	Equals Median × Total Households (redundant)
CYE Average Mean Household Income	Mean is sensitive to outliers; Median is more robust

Category D • Distance-band duplicates (~80 columns dropped)

Each competitor has 5-6 distance band columns (Trade Area, 3mi, 5mi, 10mi, 0-11mi, 11-30mi) and they are heavily correlated. We kept TradeArea only — per Lowe's own methodology, the proprietary Trade Area captures ~70% of a store's true sales catchment, making it the gold standard.

Category E · Rolling features dropped in iteration 3

roll_4 and roll_13 were initially in the 40-feature set. After the first model iteration they accounted for ~44% of feature importance combined, masking the contributions of market features. We dropped them in iteration 3, accepting a small WAPE increase (6.21 → 7.05%) in exchange for:

- Bias improvement from -2.53% to -0.97% (near-zero)
- Better explainability — the model's drivers are now interpretable to a planner
- More balanced feature importance distribution

Why these were 'less significant'

In all cases the dropped features carried duplicate or near-duplicate information. Key reasons:

1. In linear models (Ridge), multicollinearity destabilizes coefficient estimates and inflates standard errors
2. In tree models, correlated features compete for splits — one gets used, others sit idle, but they each add training cost and complicate feature-importance interpretation
3. Fewer features means simpler explanations to business stakeholders (planners must defend targets)
4. Reduces overfitting risk by lowering the model's effective degrees of freedom
5. Faster training and inference

The funnel summary

From 165 candidate features (after identity & leakage drops) to the final 16 — multicollinearity removal alone accounted for ~100 of those dropped features (distance bands + income brackets + housing decades + inverses + redundant aggregates).

Q7 • 'subsample 0.8' and 'colsample_bytree 0.8' in plain English

The intuition: building a panel of judges

Imagine you're building a panel of 500 judges to make a decision (each judge = one tree). If all 500 judges had the exact same information and saw all the same evidence, they would make highly correlated decisions — and the panel's overall judgment wouldn't be much better than one expert.

XGBoost's solution: deliberately introduce randomness so each judge gets a slightly different view of the data, producing diverse opinions that average into a more robust final answer.

subsample = 0.8

What it does

When building each individual tree, randomly select 80% of the training rows (store-week observations) and use ONLY those rows to fit that tree. The other 20% are temporarily ignored for that tree only — they may be used by the next tree.

Layman analogy

Think of it like getting feedback from 100 customers, but each focus group only invites a random 80 of them. Each focus group hears slightly different stories. The combined output across many focus groups is more representative than any single full-group session that one strong voice might dominate.

Why 0.8 specifically (not 1.0, not 0.5)

- 1.0 means every tree sees all data → trees become too similar, ensemble loses diversity benefit, overfits to training set
- 0.5 means each tree sees only half — too aggressive, each tree becomes weaker, noisier
- 0.8 is the industry-standard sweet spot — enough diversity to regularize, enough data per tree to learn well

What it prevents

Overfitting. Without subsampling, an ensemble of 500 trees can memorize quirks of the exact training data (e.g., one particular store-week that had an unusual sale). Random subsampling makes the model robust to such quirks.

colsample_bytree = 0.8

What it does

When building each individual tree, randomly select 80% of the FEATURES and consider only those features for the splits in that tree. The other 20% of features are unavailable to that tree.

Layman analogy

Imagine each judge in your panel is given only 80% of the available evidence to make their decision. Judge A might be shown lag_1, income, and competition but NOT housing share. Judge B might be shown housing share, urbanicity, and lag_4 but NOT lag_1. By forcing different judges to consider different evidence, you prevent any single piece of evidence from dominating every decision.

Why this matters HUGELY for our model specifically

Our lag_1 feature is so dominant (correlation 0.982 with target) that without colsample_bytree, every single one of the 500 trees would split on lag_1 first. Other features would never get a chance to contribute.

With colsample_bytree=0.8, lag_1 is unavailable in ~20% of trees. Those trees are FORCED to find their second-best feature — and that's exactly when market signals (housing, income, competition) get to shine. The end result: a model whose feature importance distribution is meaningful, not monopolized.

Effect on our reported feature importances

Compare what you'd see WITH vs WITHOUT colsample_bytree=0.8 (illustrative):

Feature	With colsample 1.0 (hypothetical)	With colsample 0.8 (our model)
lag_1	~92%	62.22%
lag_4	~5%	29.63%
Market features combined	~3%	~5%
Explainability	Black box (one feature drives everything)	Defensible (4% explained by interpretable market signals)

Combined effect — why both matter

Both parameters introduce randomness, but along different axes:

- subsample randomizes WHICH OBSERVATIONS each tree learns from (rows)
- colsample_bytree randomizes WHICH FEATURES each tree can use (columns)

Together they make the 500-tree ensemble truly diverse — and reduced overfitting is the result. Both are forms of 'regularization through randomness'.

Bottom line for the talking point

"subsample 0.8 means each tree only sees 80% of the rows, randomly chosen. colsample_bytree 0.8 means each tree only considers 80% of the features. Both inject randomness so the 500 trees in our ensemble are diverse instead of identical. The end result is a model that generalizes well to new data and doesn't let any single feature dominate every decision."

